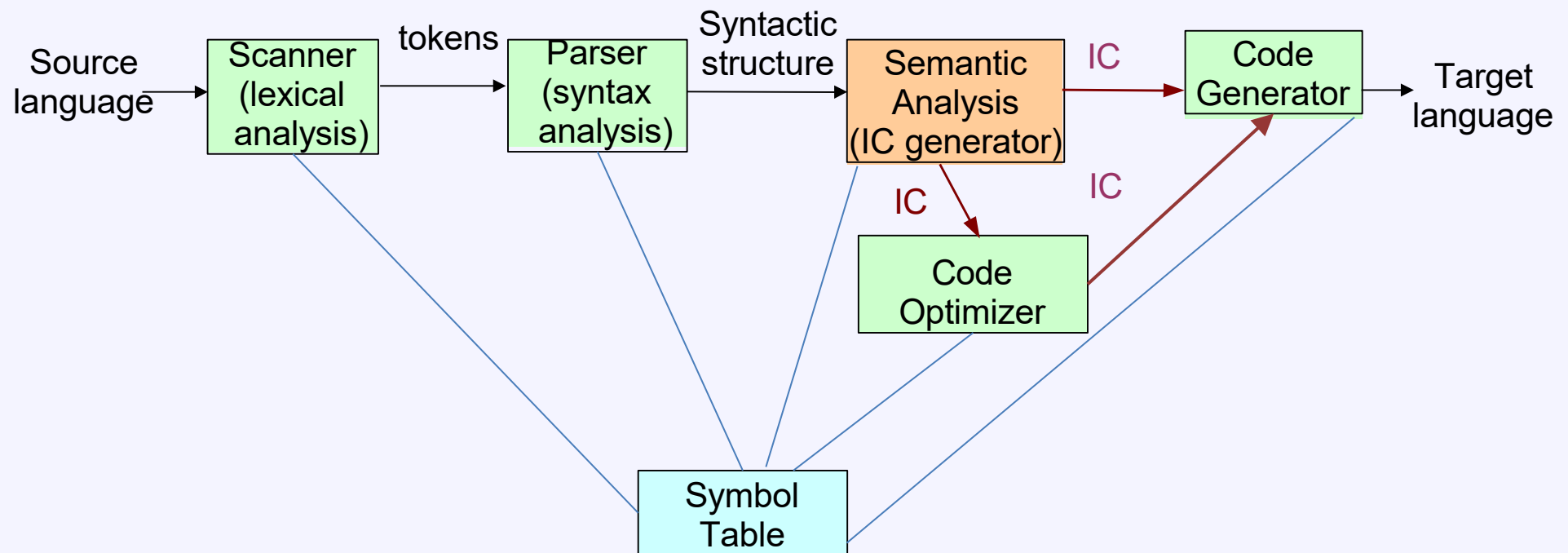


Generovanie do medzijazyka

Ján Šturc, Richard Ostertág,
Jana Kostičová

- Formy medzijazyka
- Generovanie výrazov
- Booleovské výrazy
- Príkazy
- Volania
- Spätné plátanie

Použitie medzijazyka v kompilátore



Formy medzijazyka

Poľská sufixová forma (Reverse Polish Notation)

- Hlavne pre výrazy
- Výnimočne len pre malé zariadenia. Neumožňuje prakticky žiadnu optimalizáciu. Pre bezprostredné vyhodnotenie na zásobníkovom automate (napr. kalkulačky)

výraz: $a := b * c + b * c$

postfix: $abc*bc*+ :=$

Syntaktické stromy a DAGy

- DAG – spoločné podvýrazy sa počítajú iba raz.

Trojadresový kód

- Najviac jeden operátor na pravej strane inštrukcie
- Štvorice, trojice, nepriame trojice
- Štvorice – v súčasnosti najpoužívanější tvar medzijazyka

Navzájom
súvisia

Syntaktické stromy a DAGy

- Grafická reprezentácia (binárne stromy), ktoré obsahujú:
 - binárne vnútorné uzly – nodes (operácia a dva smerníky na operandy)
 - unárne vnútorné uzly – unodes (operácia a smerník na operand)
 - listy – leaf obsahujú identifikátory a ich hodnoty
- Sémantické procedúry
 - `mknnode(op:operation, left, right: pointer):pointer`
 - `mkunode(op:operation, child: pointer):pointer`
 - `mkleaf("id", id.place:pointer):pointer`
- Lineárna reprezentácia sú trojice (štvorice)
 - unode má druhý smerník prázdny
 - leaf má v prvej pološke „identifikátor“
- Lineárnu reprezentáciu vytvoríme očíslovaním trojíc (štvoríc) pomocou post-order traverzovania.

Syntaxou riadený preklad

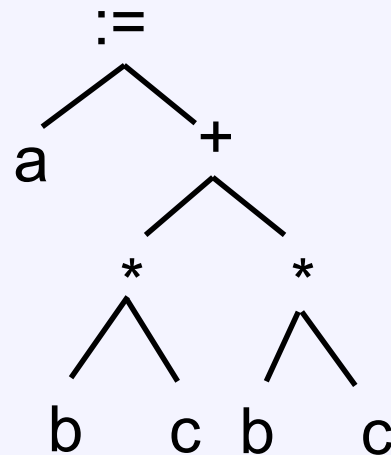
- SRD - môže byť použitá na konštrukciu syntaktického stromu aj DAGu

Syntax	Sémantika
$S \rightarrow \mathbf{id} := E$	$S.nptr := \text{mknnode}(":=", \text{mkleaf}(\mathbf{id}, \mathbf{id.place}), E.nptr)$
$E \rightarrow E_1 + E_2$	$E.nptr := \text{mknnode}("+", E_1.nptr, E_2.nptr)$
$E \rightarrow E_1 * E_2$	$E.nptr := \text{mknnode}("*", E_1.nptr, E_2.nptr)$
$E \rightarrow - E_1$	$E.nptr := \text{mkunode}("uminus", E_1.nptr)$
$E \rightarrow (E_1)$	$E.nptr := E_1.nptr$
$E \rightarrow \mathbf{id}$	$E.nptr := \text{mkleaf}(\mathbf{id}, \mathbf{id.place})$

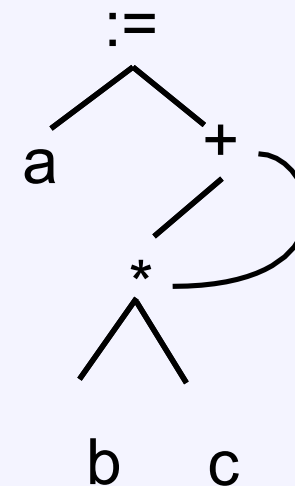
Príklad

výraz: $a := b * c + b * c$

strom výrazu:



dag výrazu:



p1 = mkleaf(id, entry-b)

p2 = mkleaf(id, entry-c)

p3 = mkleaf(id, entry-b) = p1

p4 = mkleaf(id, entry-c) = p2

p5 = mknnode(*, p1, p2)

p6 = mknnode(*, p3, p4) = p5

p7 = mknnode(*, p6, p7)

p8 = mknnode(*, p1, p8)

p9 = mkleaf(id, entry-a)

p10 = mknnode(:=, p9, p8)

- Kontrola signatúry
<op, l, r >
- Implementácia: napr.
hashovacia tabuľka

Trojadresový kód

- Najviac jeden operátor na pravej strane inštrukcie
- „Adresy“:
 - meno (nahradené smerníkom do tabuľky symbolov)
 - konštanta
 - dočasné mená (generované)
- Inštrukcie
 - aritmetické, logické
 - $+$, $-$, $u-$, $*$, $/$, (real, integer, fixed), div, mod
 - $\neg$, $\wedge$, $\vee$, $\oplus$, $<<$, $>>$, (rôzne shifty)
 - move (priradenie, zapamätanie, skoky)
 - **if** op1 ($<$, $>$, $=$, $\neq$) 0 **then goto** op2;
 - Ak vieme adresovať všetky registre, skok je move to PC (program counter).
 - indirekcia a referencia (A[i], &a)
 - result:= $*(\text{op1} + \text{op2})$
 - result:= &op2
- Nebudeme v ďalšom príliš dodržiavať formalizmus. Zápis v štýle publikácie programov.

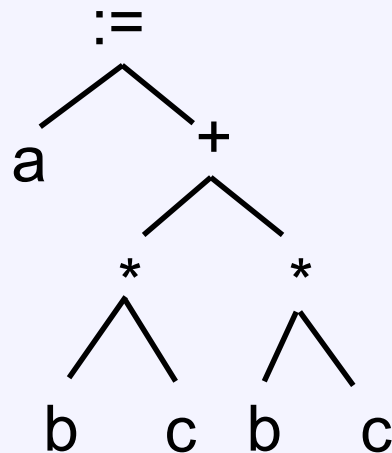
„Dračí“ medzijazyk

1. $x := y \text{ op } z$	assignment
2. $x := \text{op } y$	unary assignment
3. $x := y$	copy
4. goto L	unconditional jump
5. if ($x \text{ relop } y$) goto L	conditional jump
6. param x	procedure call
7. call p n	procedure call
8. return y	procedure call
9. $x := y[i]$	indexed assignment
10. $x[i] := y$	indexed assignment

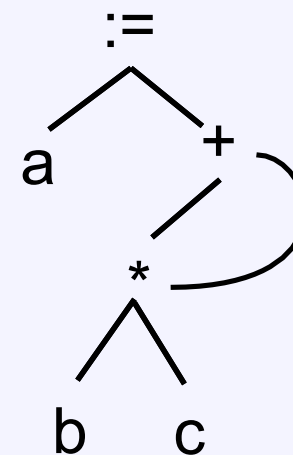
Príklad

výraz: $a := b * c + b * c$

strom výrazu:



dag výrazu:



Trojadresový kód =
lineárna
reprezentácia
(dočasné mená
zodpovedajú
vnútorným uzlom)

```
t1 := b * c  
t2 := b * c  
t3 := t1 + t2  
a := t3
```

```
t1 := b * c  
t2 := t1  
t3 := t1 + t2  
a := t3
```

Reprezentácia

Štvorice: Program je postupnosť záznamov tvaru:

```
struct { oper: operation;  
        address: operand1, operand2,result}
```

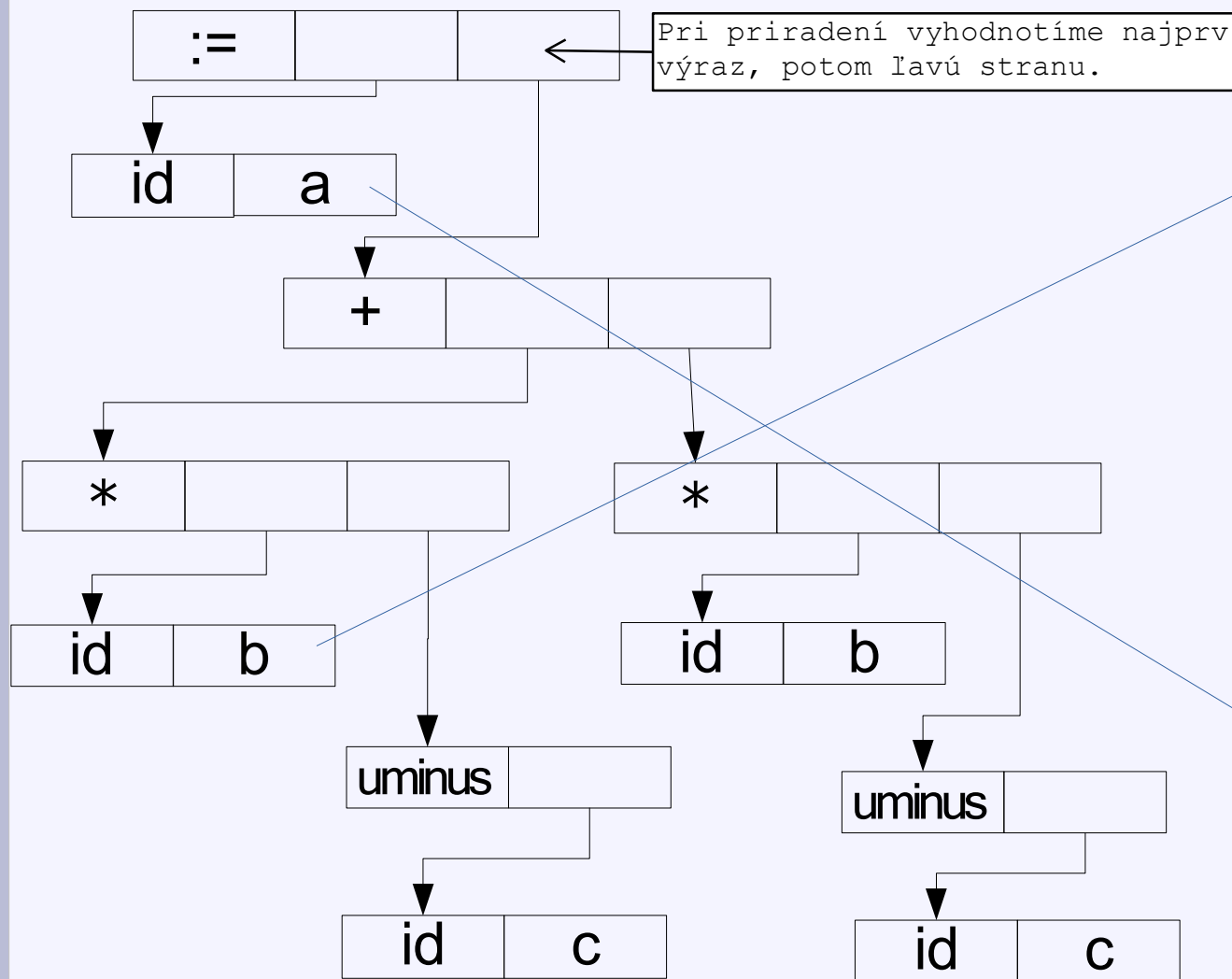
Trojice

- Šetria 25% pamäte
Pre získanie výsledku sa „adresuje“ trojica.
„Absolútna adresácia“ kódu sťažuje optimalizácie.

Nepriame trojice

- Kompromis. Tabuľka pre nepriamu adresáciu trojíc umožňuje ich preusporiadanie.

Trojice: strom $a := b * (-c) + b * (-c)$



Trojice

0	id	b	
1	id	c	
2	u-	1	
3	*	0	2
4	id	b	
5	id	c	
6	u-	5	
7	*	4	6
8	+	3	7
9	id	a	
10	:=	9	8

Nepriame trojice

35	(0)
36	(1)
37	(2)
38	(3)
39	(4)

Zoznam inštrukcií

Trojice

0	id	b	
1	id	c	
2	u-	1	
3	*	0	2
4	id	b	
5	id	c	
6	u-	5	
7	*	4	6
8	+	3	7
9	id	a	
10	:=	9	8

Spôsoby prirad'ovania labelov

do $i = i + 1$; while ($a[i] < v$);

L: $t_1 = i + 1$

$i = t_1$

$t_2 = i * 8$

$t_3 = a[t_2]$

if $t_3 < v$ goto **L**

100: $t_1 = i + 1$

101: $i = t_1$

102: $t_2 = i * 8$

103: $t_3 = a[t_2]$

104: if $t_3 < v$ goto **100**

Symbolické mená vs indexy inštrukcií

Generovanie trojadresového kódu

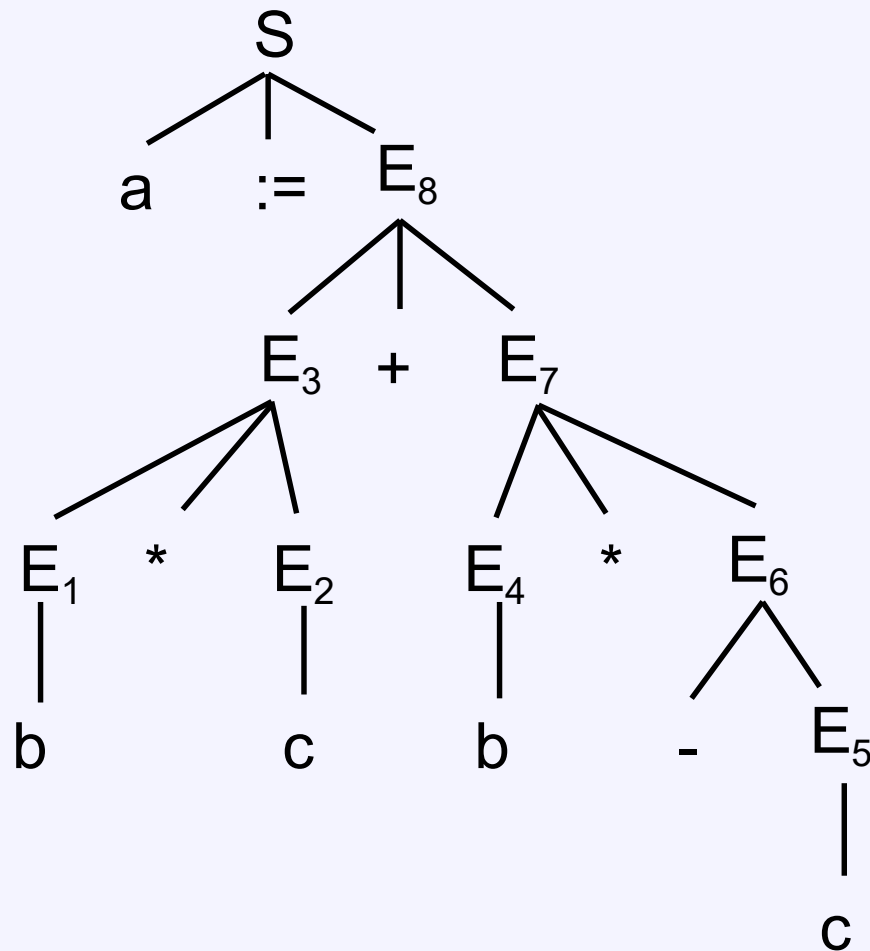
- Na generovanie trojadresového kódu použijeme syntaxou riadený preklad.
- Začneme jazykom pozostávajúcim z priradenia a výrazov.
- Príkaz priradenia S má jediný atribút **code** - vygenerovaný kód.
- Výrazy E majú dva atribúty:
 - **code** – úsek kódu zodpovedajúci výrazu
 - **place** – meno premennej, v ktorej je uložená hodnota výrazu E .
- Označenie: $gen(x \text{ ":=" } y \text{ "+" } z)$ reprezentuje príkaz $x := y + z$
- Označenie $\langle fragment \rangle || expr$ znamená zreťazenie fragmentu kódu s výrazom.

Syntaxou riadený preklad

```
S → id := E    { S.code := E.code || gen(id.place "!=" E.place) }
E → E1 + E2    { E.place := newtemp;
                  E.code := E1.code || E2.code ||
                  gen(E.place "!=" E1.place "+" E2.place)}
E → E1 * E2    { E.place := newtemp;
                  E.code := E1.code || E2.code ||
                  gen(E.place "!=" E1.place "*" E2.place) }
E → - E1        { E.place := newtemp;
                  E.code := E1.code ||
                  gen(E.place "!=" "uminus" E1.place) }
E → (E1)        { E.place := newtemp;
                  E.code := E1.code || gen(E.place "!=" E1.place) }
E → id          { E.place := id.place;
                  E.code := "" }
```

Príklad – syntaktický strom

$a := b * c + b * -c$

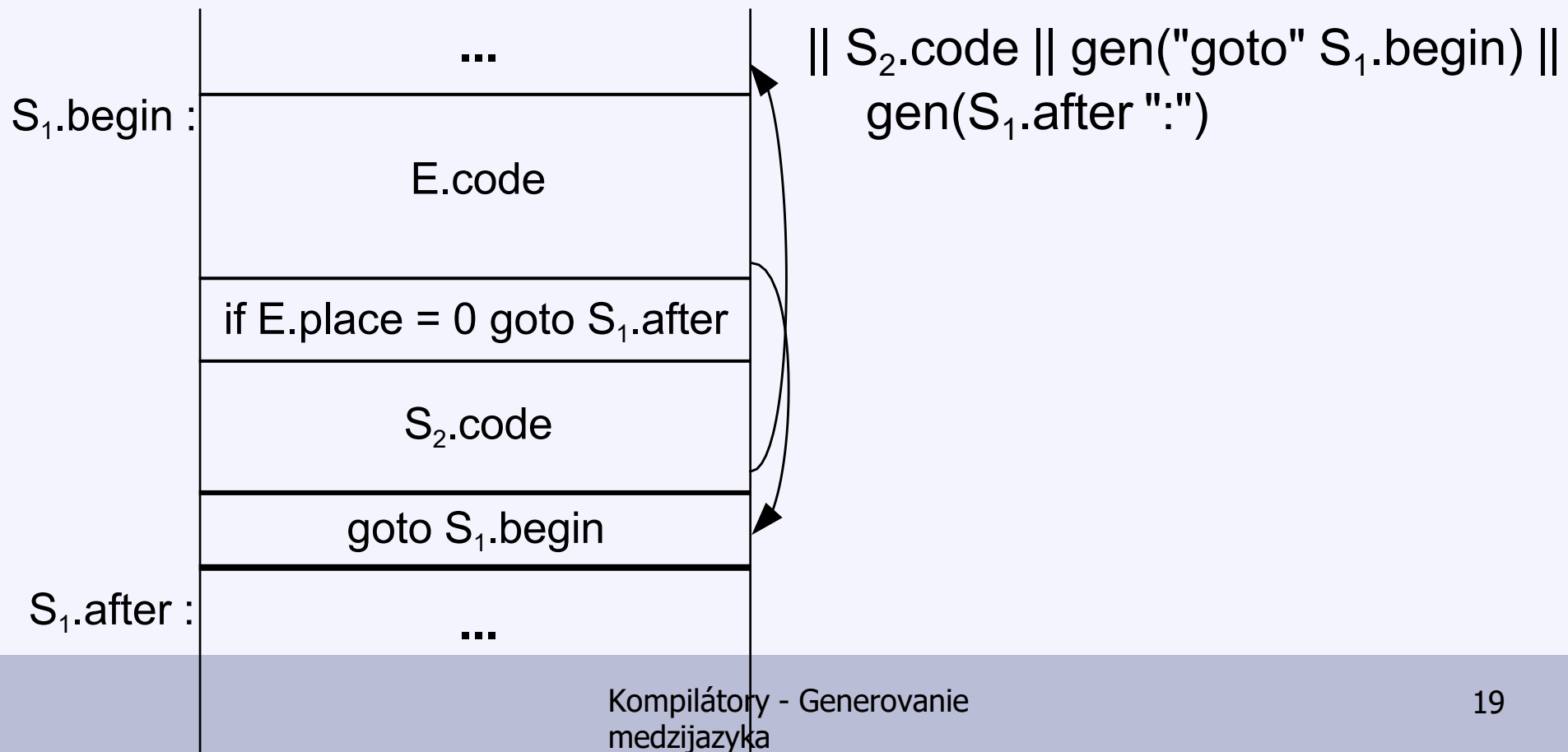


Príklad – generovanie kódu

	place	code
E_1	b	
E_2	c	
E_3	t_1	$E_1.\text{code} \parallel E_2.\text{code} \parallel t_1 := b * c$
E_4	b	
E_5	c	
E_6	t_2	$E_5.\text{code} \parallel t_2 := u - c$
E_7	t_3	$E_4.\text{code} \parallel E_6.\text{code} \parallel t_3 := b * t_2$
E_8	t_4	$E_3.\text{code} \parallel E_7.\text{code} \parallel t_4 := t_1 + t_3$
S		$E_8.\text{code} \parallel a := t_4$

Tok riadenia

$S_1 \rightarrow \text{while } E \text{ do } S_2$
 $S_1.\text{begin} := \text{newlabel};$
 $S_1.\text{after} := \text{newlabel};$
 $S_1.\text{code} := \text{gen}(S_1.\text{begin} ":") \parallel E.\text{code} \parallel$
 $\text{gen}(\text{"if" } E.\text{place} \text{ "=" } 0 \text{ goto" } S_1.\text{after})$



Deklarácie

- Z typu mena vieme určiť, koľko pamäti preňho potrebujeme (*type width*), napr.
 - Integer -> 4 bytes
 - Real -> 8 bytes
 - Array[num] of T -> num * veľkosť typu T
- Vypočítame adresu relatívnu vzhľadom na začiatok bloku
- Typ (type) a relatívnu adresu (offset) uložíme v tabuľke symbolov pre dané meno:
id, type, offset
- Dáta premenlivej dĺžky (reťazce, dynamické polia) -> rezervuje sa množstvo pamäte potrebné pre uloženie smerníka
- Zarovnanie adresy (address alignment)
 - Napr. integer musí byť uložený na adrese deliteľnej 4mi
 - Efektívnejší prístup, niektoré architektúry ho vyslovene vyžadujú

Deklarácie

$P \rightarrow \textcolor{red}{M}D;B.$

Značkovací (narker)
netermínál

$M \rightarrow \varepsilon$

$D \rightarrow D;D$

$D \rightarrow \text{id}:T$

Premenná s nasledujúcou
dostupnou adresou

$\{ \textcolor{red}{offset}:=0 \}$

Vloží záznam do tabuľky
symbolov

$\{ \textcolor{red}{enter}(\text{id.name}, T.\text{type}, \text{offset});$
 $\text{offset}:= \text{offset} + T.\text{width} \quad \}$

$T \rightarrow \text{integer}$

$\{ T.\text{type}:= \text{integer}; T.\text{width}:= 4 \quad \}$

$T \rightarrow \text{real}$

$\{ T.\text{type}:= \text{real}; T.\text{width}:= 8 \quad \}$

$T \rightarrow \text{array}[\text{num}] \text{ of } T_1$

$\{ T.\text{type}:= \text{array}(\text{num.val}, T_1.\text{type});$
 $T.\text{width}:= \text{num.val} * T_1.\text{width} \}$

$T \rightarrow \uparrow T_1$

$\{ \textcolor{red}{T.type}:= \text{pointer}(T_1.\text{type});$
 $\textcolor{red}{T.width}:= 4 \quad \}$

Syntetizované
atribúty

Pozn. Práca s tabuľkou symbolov je trochu komplikovanejšia Treba zohľadniť „scope“ deklarácií.

Záznamy (records)

Príklad:

a record

x: array[3] of integer;

y: real;

z ↑ integer;

end

...
*a, record(**t**), offset*
...



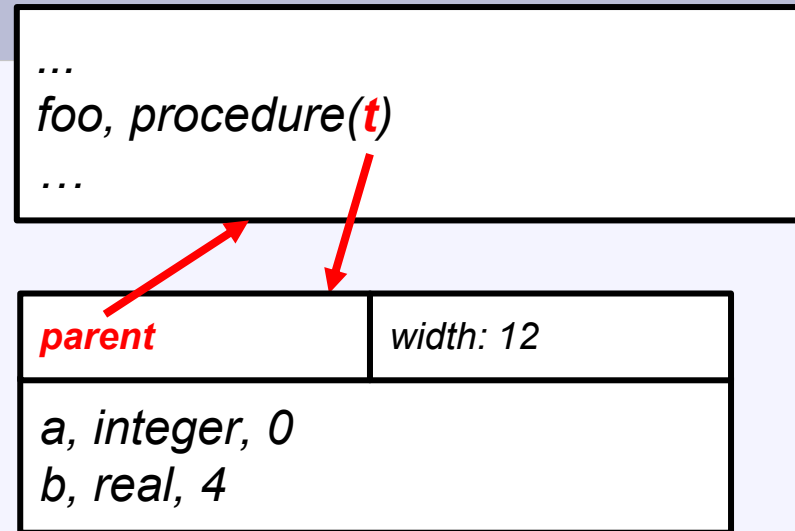
parent: NIL	width: 24
<i>x, array(3, integer), 0</i> <i>y, real, 12</i> <i>z, pointer(integer), 20</i>	

- Samostatná tabuľka symbolov pre každý záznamový typ
 - Slúži na informáciu o type (celková šírka, adresácia položiek)
- Adresy položiek záznamu sa počítajú relatívne vzhľadom na dátovú oblasť tohto záznamu
- Ak má meno ako typ záznam, v tabuľke symbolov má tento typ formu
record(t)
kde *t* je smerník do tabuľky symbolov pre daný záznam
- Podobný prístup funguje aj pre triedy

Procedúry

Príklad:

```
proc foo ;  
  a: integer;  
  b: real;  
  ....  
end
```



- Samostatná tabuľka symbolov pre každú procedúru
 - Vnorený scope, smerník na nadradenú tabuľku
- Adresy mien v procedúre sa počítajú relatívne vzhľadom na dátovú oblasť procedúry
- Ak máme definíciu procedúry, v tabuľke symbolov má tento typ formu

procedure(t)

kde *t* je smerník do tabuľky symbolov pre danú procedúru

„Dračie“ sémantické procedúry

1. `mktable(previous: pointer): pointer`

Vytvára novú tabuľku symbolov. Argumentom je smerník na nadradenú tabuľku a vracia smerník na novo vytvorenú tabuľku.

2. `enter(table:pointer, name:string, type:type, offset:int)`

Vytvára nový záznam v tabuľke symbolov pre identifikátor `name`.

3. `addwidth(table:pointer, width:integer)`

Zaznamená kumulatívnu dĺžku všetkých položiek v hlavičke tabuľky symbolov.

4. `enterproc(table:pointer, name:string, newtable:pointer)`

Vytvára novú položku pre vnorenú procedúru v danej tabuľke. **Newtable** ukazuje na novovytvorenú tabuľku symbolov pre túto procedúru.

Použitie

$P \rightarrow M D ; B.$	$\{ \text{addwidth}(\text{top}(\text{tblptr}), \text{top}(\text{offset}));$ $\text{pop}(\text{tblptr}); \text{pop}(\text{offset}) \}$	
$M \rightarrow \varepsilon$	$\{ t := \text{mktable}(\text{nil});$ $\text{push}(t, \text{tblptr}); \text{push}(0, \text{offset}) \}$	„Globálna“ tabuľka
$D \rightarrow D_1 ; D_2$		
$D \rightarrow \text{proc id} ; N D_1 ; S \text{ end}$	$\{ t := \text{top}(\text{tblptr}); \text{addwidth}(t, \text{top}(\text{offset}));$ $\text{pop}(\text{tblptr}); \text{pop}(\text{offset});$ $\text{enterproc}(\text{top}(\text{tblptr}), \text{id.name}, t) \}$	
$D \rightarrow \text{id} : T$	$\{ \text{enter}(\text{top}(\text{tblptr}), \text{id.name}, T.\text{type}, \text{top}(\text{offset}));$ $\text{top}(\text{offset}) := \text{top}(\text{offset}) + T.\text{width} \}$	
$N \rightarrow \varepsilon$	$\{ t := \text{mktable}(\text{top}(\text{tblptr}));$ $\text{push}(t, \text{tblptr}); \text{push}(0, \text{offset}) \}$	Nová tabuľka pre procedúru
$T \rightarrow \text{record } L D \text{ end}$	$\{ T.\text{type} := \text{record}(\text{top}(\text{tblptr}));$ $T.\text{width} := \text{top}(\text{offset});$ $\text{pop}(\text{tblptr}); \text{pop}(\text{offset}); \}$	
$L \rightarrow \varepsilon$	$\{ t := \text{mktable}(\text{nil});$ $\text{push}(t, \text{tblptr}); \text{push}(0, \text{offset}) \}$	Nová tabuľka pre záznamový typ

Príkaz priradenia

```
S → id := E      { p:= lookup(id.name);  
                    if p ≠ nil then gen(p ":=" E.place) else error      }  
  
E → E1 + E2    { E.place := newtemp;  
                    gen(E.place ":=" E1.place "+" E2.place)      }  
  
E → E2 * E3    { E.place := newtemp;  
                    gen(E.place ":=" E1.place "*" E2.place)      }  
  
E → - E1        { E.place := newtemp;  
                    gen(E1.place ":=" uminus" E1.place      }  
  
E → (E1)        { E.place:= E1.place      }  
  
E → id          { p:= lookup(id.name);  
                    if p ≠ nil then E.place := p else error      }
```

Polia (arrays)

Polia ukladáme súvisle prvok po prvku: $A[\text{low}:\text{high}]$ nech w je veľkosť jedného prvku. Potom adresa i -tého prvku je:

$$\text{base} + (i - \text{low}) * w = \text{base} - \text{low} * w + i * w = A[0] + i * w$$

Pre dvojrozmerné polia, adresa prvku $A[i_1, i_2]$ je

$$A[0,0] + i_1 * w_1 + i_2 * w_2$$

Alternatívne ak n_2 je počet prvkov druhej dimenzie a $w = w_2$ (šírka typu prvkov poľa)

$$\begin{aligned} & A[0,0] + i_1 * (w * n_2) + i_2 * w = \\ & = A[0,0] + (i_1 * n_2 + i_2) * w \end{aligned}$$

Polia (arrays)

Zovšeobecnenie pre mnohorozmerné polia. Nech

$$n_j = \text{high}_j - \text{low}_j + 1 \text{ (počet prvkov v dimenzii j)}$$

pre $j \leq k$. Adresa prvku $A[i_1, i_2, \dots, i_k]$ je:

$$A[0, 0, \dots, 0] + ((\dots(i_1 * n_2 + i_2) * n_3 + i_3) \dots) * n_k + i_k) * w,$$

kde

$$A[0, 0, \dots, 0] = \text{base} - ((\dots(\text{low}_1 * n_2 + \text{low}_2) * n_3 + \text{low}_3) \dots) * n_k + \text{low}_k) * w.$$

Base ukazuje na prvé voľné miesto na zásobníku. Kvôli tomu musíme počítať fiktívnu adresu nultého prvku. Označíme ju $c(A)$. Ak dovolíme len statické polia, stačí ju počítať počas kompilácie.

Polia (arrays)

Neterminál L generuje jednoduchú premennú alebo prvok poľa

$L \rightarrow Elist]$ *// prvok poľa: id + postupnosť výrazov pre indexy*

$L \rightarrow id$ *// jednoduchá premenná: id*

$Elist \rightarrow Elist_1, E$ *// pridaj ďalší index*

$Elist \rightarrow id [E$ *// ukončenie – meno poľa + prvý index*

Linearizácia indexov (row-major)

$id [t_1, t_2, t_3, \dots] \Rightarrow id [t]$

*(po pre násobení šírkou prvku poľa w dostaneme offset: $offset = i * w$)*

Nech $Elist$ reprezentuje i -ty index

$Elist.array$ - ukazuje na záznam v tabuľke symbolov pre pole id

$Elist.dim$ = poradové číslo (pozícia) naposledy spracovaného indexu

$Elist.place = id [t]$, kde t obsahuje linearizovaný index pre $id [t_1, t_2, \dots t_{i-1}]$

Syntaxou riadený preklad

```

S → L := E    { if L.offset = null then gen(L.place ":=" E.place)
                iba id, nie pole → else gen(L.place "[" L.offset "]" := E.place) }
E → L          { if L.offset = null then E.place = L.place
                else E.place := newtemp;
                gen(E.place ":=" L.place "[" L.offset "]" ) }
L → Elist ]    { L.place := newtemp; L.offset := newtemp;
                gen(L.place ":=" c(Elist.array));
                vracia w → gen(L.offset ":=" Elist.place "*" width(Elist.array)) }
L → id         { L.place := id.place; L.offset := null
                limit vracia nj z tab. sym. vytvorenej pri deklarácii pola }
Elist → Elist1, E { t := newtemp; dim := Elist1.ndim + 1;
                  gen(t ":=" Elist1.place "*" limit(Elist1.array, dim));
                  gen(t ":=" t "+" E.place); Elist.array := Elist1.array;
                  Elist.place := t; Elist.ndim := dim }
Elist → id [ E { Elist.array := id.place;
                  Elist.place := E.place; Elist.ndim := 1 }

```

Vynutená konverzia (coercion)

Relatívne úplná semantika pravidla $E \rightarrow E_1 + E_2$

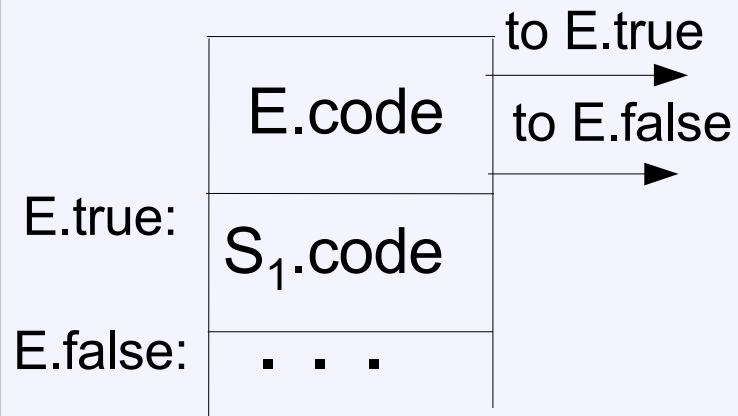
```
E.place := newtemp; /* Radšej do každej vetvy ako druhý príkaz. */  
if  $E_1.type = \text{integer}$  and  $E_2.type = \text{integer}$  then  
    {  $E.type := \text{integer}$ ; gen( $E.place := E_1.place \text{ "int + " } E_2.place$ ) }  
else if  $E_1.type = \text{real}$  and  $E_2.type = \text{real}$  then  
    {  $E.type := \text{real}$ ; gen( $E.place := E_1.place \text{ "real + " } E_2.place$ ) }  
else if  $E_1.type = \text{integer}$  and  $E_2.type = \text{real}$  then  
    {  $E.type := \text{real}$ ;  $u := \text{newtemp}$ ; gen( $u := \text{ "intoreal" } E_1.place$ );  
      gen( $E.place := u \text{ "real + " } E_2.place$ ) }  
else if  $E_1.type = \text{real}$  and  $E_2.type = \text{integer}$  then  
    {  $E.type := \text{real}$ ;  $u := \text{newtemp}$ ; gen( $u := \text{ "intoreal" } E_2.place$ );  
      gen( $E.place := E_1.place \text{ "real + " } u$ ) }  
else  $E.type := \text{type\_error}$ ;
```

Tok riadenia

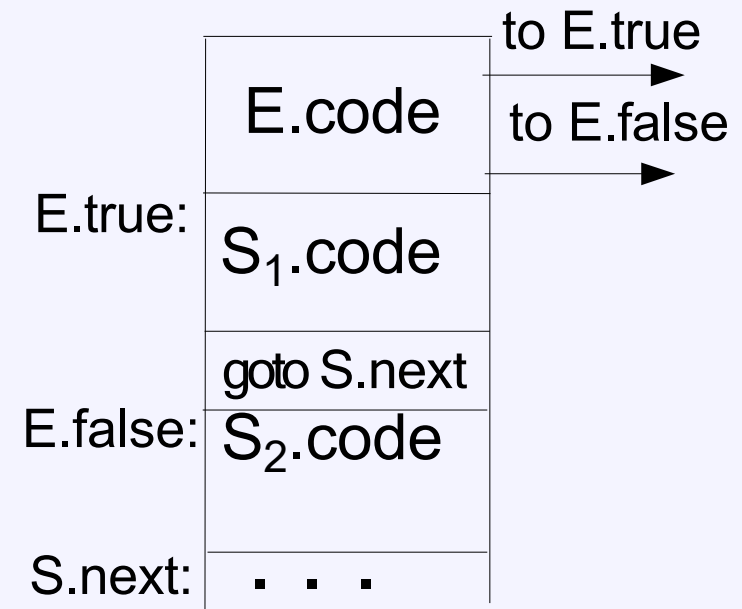
$S \rightarrow \text{if } E \text{ then } S_1$	$\{ E.\text{true} := \text{newlabel}; E.\text{false} := S.\text{next};$ $S_1.\text{next} := S.\text{next};$ $S.\text{code} := E.\text{code} \parallel \text{gen}(E.\text{true} ":") \parallel$ $S_1.\text{code}$ $\}$
$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$	$\{ E.\text{true} := \text{newlabel}; E.\text{false} := \text{newlabel};$ $S_1.\text{next} := S.\text{next}; S_2.\text{next} := S.\text{next};$ $S.\text{code} := E.\text{code} \parallel \text{gen}(E.\text{true} ":") \parallel$ $S_1.\text{code} \parallel \text{gen}(\text{"goto"} S.\text{next}) \parallel$ $\text{gen}(E.\text{false} ":") \parallel S_2.\text{code}$ $\}$
$S \rightarrow \text{while } E \text{ do } S_1$	$\{ S.\text{begin} := \text{newlabel}; E.\text{true} := \text{newlabel};$ $E.\text{false} := S.\text{next}; S_1.\text{next} := S.\text{begin};$ $S.\text{code} := \text{gen}(S.\text{begin} ":") \parallel E.\text{code} \parallel$ $\text{gen}(E.\text{true} ":") \parallel S_1.\text{code} \parallel$ $\text{gen}(\text{"go to"} S.\text{begin})$ $\}$

Grafické znázornenie

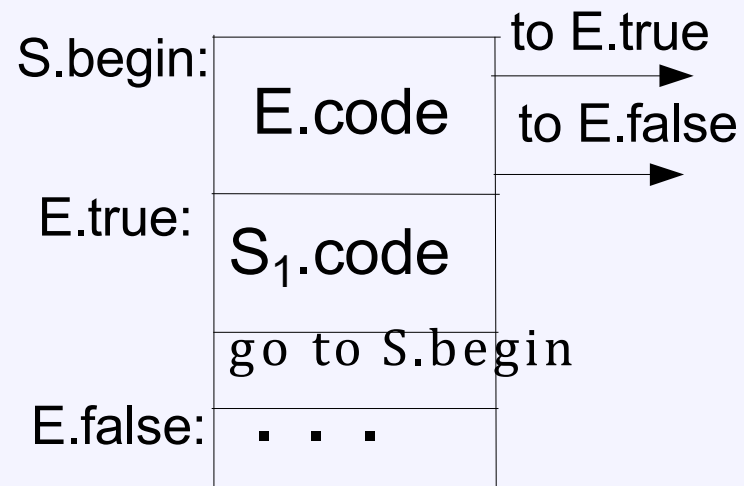
if then príkaz



if then else príkaz



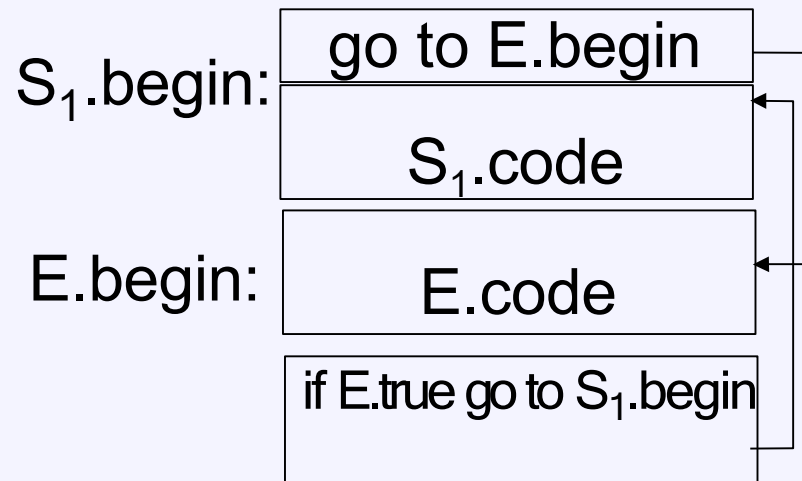
while príkaz



Iná šablóna pre while príkaz

$S \rightarrow \text{while } E \text{ do } S_1 \{ S_1.\text{begin} := \text{newlabel}; E.\text{begin} := \text{newlabel}; S.\text{code} := \text{gen}(\text{"go to" } S_1.\text{begin}) \parallel$

$S_1.\text{code} \parallel E.\text{code} \parallel$
 $\text{gen}(\text{"if } E.\text{true go to" } S_1.\text{begin}) \}$



Pokiaľ boolovské výrazy počítame aritmetickým spôsobom, je táto šablóna výhodnejšia.

Booleovské výrazy 1 – klasika

$E \rightarrow E_1 \text{ or } E_2$	<pre>{ E.place := newtemp; gen(E.place ":=" E₁.place "or" E₂.place) }</pre>
$E \rightarrow E_1 \text{ and } E_2$	<pre>{ E.place := newtemp; gen(E.place ":=" E₁.place "and" E₂.place) }</pre>
$E \rightarrow \text{not } E_1$	<pre>{ E.place := newtemp; gen(E.place ":=" "not" E₁.place) }</pre>
$E \rightarrow (E_1)$	<pre>{ E.place := E₁.place }</pre>
$E \rightarrow \text{true}$	<pre>{ E.place := newtemp; gen(E.place ":=" "1") }</pre>
$E \rightarrow \text{false}$	<pre>{ E.place := newtemp; gen(E.place ":=" "0") }</pre>
$E \rightarrow aE_1 \text{ relop } aE_2$	<pre>{ E.place := newtemp; gen("if" aE₁.place relop aE₂.place "go to" +3); gen(E.place ":=" "0"); gen("go to" +2); gen(E.place ":=" "1") }</pre>

Boolovské výrazy 2 – skratkou

$E \rightarrow E_1 \text{ or } E_2$	$\{ E_1.\text{true} := E.\text{true}; E_1.\text{false} := \text{newlabel};$ $E_2.\text{true} := E.\text{true}; E_2.\text{false} := E.\text{false};$ $E.\text{code} := E_1.\text{code} \parallel \text{gen}(E_1.\text{false} ":") \parallel E_2.\text{code} \}$
$E \rightarrow E_1 \text{ and } E_2$	$\{ E_1.\text{true} := \text{newlabel}; E_1.\text{false} := E.\text{false};$ $E_2.\text{true} := E.\text{true}; E_2.\text{false} := E.\text{false};$ $E.\text{code} := E_1.\text{code} \parallel \text{gen}(E_1.\text{true} ":") \parallel E_2.\text{code} \}$
$E \rightarrow \text{not } E_1$	$\{ E_1.\text{true} := E.\text{false}; E_1.\text{false} := E.\text{true};$ $E.\text{code} := E_1.\text{code} \}$
$E \rightarrow (E_1)$	$\{ E_1.\text{true} := E.\text{true}; E_1.\text{false} := E.\text{false};$ $E.\text{code} := E_1.\text{code} \}$
$E \rightarrow \text{true}$	$\{ \text{gen}(\text{"go to" } E.\text{true}) \}$
$E \rightarrow \text{false}$	$\{ \text{gen}(\text{"go to" } E.\text{false}) \}$
$E \rightarrow aE_1 \text{ relop } aE_2$	$\{ E.\text{code} := \text{gen}(\text{"if" } aE_1.\text{place relop } aE_2.\text{place}$ $\text{"go to" } E.\text{true}) \parallel \text{gen}(\text{"go to" } E.\text{false}) \}$

Príkaz case – šablóny

case E of

$V_1: S_1$

$V_2: S_2$

...

$V_{n-1}: S_{n-1}$

default: S_n

end

$t := E$

go to test

$L_1: S_1.\text{code}$

go to next

...

$L_{n-1}: S_{n-1}.\text{code}$

go to next

$L_n: S_n.\text{code}$

go to next

test: if $t = V_1$ go to L_1

...

if $t = V_{n-1}$ goto L_{n-1}
go to L_n

next:

Dvojice (V_1, L_1)
sú uložené
do fronty

$t := E$

if $t \neq V_1$ go to L_1

$S_1.\text{code}$

go to next

$L_1:$ if $t \neq V_2$ go to L_2

$S_2.\text{code}$

go to next

$L_2:$

...

$L_{n-2}: \text{if } t \neq V_{n-1} \text{ go to } L_{n-1}$

$S_{n-1}.\text{code}$

go to next

$L_{n-1}: S_n.\text{code}$

next:

Dvojice (V_1, L_1)
sú generované
z fronty

Volanie procedúr

```
S → call id (L) {  
    foreach p in queue: gen("param " p)  
    gen("call " id.place)  
}
```

```
L → L, E { insert E.place in the queue }
```

```
L → E {  
    initialize queue  
    insert E.place in the queue  
}
```

Spätné plátanie – backpatching

- Problémom sú dopredné skoky na neznáme ciele
- Najjednoduchší spôsob implementácie dva prechody
 - Vygenerovať (anotated) syntaktický strom
 - Generovať kód pomocou depth first traversingu tohto stromu
- Pri jednoprechodovom generovaní sú problémom dopredné skoky toku riadenia. Najjednoduchšie je nechať ciele skoku prázdne, poznamenať si ich do nejakého zoznamu a doplniť až keď príslušná adresa je vygenerovaná. Používame príkazy:
 - `makelist(i:index_to_quad):pointer_to_list` Vytvorí zoznam obsahujúci index `i` – poradové číslo štvorice a vráti smerník na tento zoznam.
 - `merge(p1, p2:pointer_to_list):pointer_to_list` Spojí dva zoznamy a vráti smerník na výsledný zoznam.
 - `backpatch(p:pointer_to_list, a:index_to_quad)` Dosadí adresu `a` do všetkých štvoríc v zozname, na ktorý ukazuje `p`.

Príklad 1 – boolovské výrazy

$E \rightarrow E_1 \text{ or } M E_2$	<pre>{ backpatch(E₁.falselist, Mquad); E.truelist:= merge(E₁.truelist, E₂.truelist); E.falselist:= E₂.falselist }</pre>
$E \rightarrow E_1 \text{ and } M E_2$	<pre>{ backpatch(E₁.truelist, Mquad); E.truelist:= E₂.truelist; E.falselist:= merge(E₁.falselist, E₂.falselist) }</pre>
$E \rightarrow \text{not } E_1$	<pre>{ E.truelist:= E₁.falselist; E.falselist:= E₁.truelist }</pre>
$E \rightarrow (E_1)$	<pre>{ E.truelist:= E₁.truelist; E.falselist:= E₁.falselist }</pre>
$E \rightarrow \text{true}$	<pre>{ E.truelist:= makelist(nextquad); gen("go to" _) }</pre>
$E \rightarrow \text{false}$	<pre>{ E.falselist:= makelist(nextquad); gen("go to" _) }</pre>
$E \rightarrow aE_1 \text{ relop } aE_2$	<pre>{ E.truelist:= makelist(nextquad); E.falselist:= makelist(nextquad+1); gen("if" aE₁.place relop.op aE₂.place "go to" _); gen("go to" _) }</pre>
$M \rightarrow \varepsilon$	<pre>{ M.quad:= nextquad }</pre>

Príklad 2 – tok riadenia

```
S → if E then M1 S1 N else M2 S2
    { backpatch(E.truelist, M1.quad);
      backpatch(E.falselist, M2.quad);
      S.nextlist:= merge(S1.nextlist, merge(N.nextlist, S2.nextlist ) }

N → ε      { N.nextlist:= makelist(nextquad); gen("go to" _ )      }
M → ε      { M.quad:=  nextquad                                     }
```



```
S → if E then M S1  { backpatch(E.truelist, M.quad);
                        S.nextlist:= merge(E.falselist, S1.nextlist) }
```



```
S → while M1 E do M2 S1 { backpatch(S1.nextlist, M1.quad);
                                backpatch(E.truelist, M2.quad);
                                S.nextlist:= E.falselist;
                                gen("go to" M1.quad)          }
```


Príklad 2 – dokončenie

$S \rightarrow \text{begin } L \text{ end}$ { $S.\text{nextlist} := L.\text{nextlist}$ }

$S \rightarrow A$ { $S.\text{nextlist} := \text{null}$ }



$L \rightarrow L_1 ; M S$ { $\text{backpatch}(L_1.\text{nextlist}, M.\text{quad});$
 $L.\text{nextlist} := S.\text{nextlist}$ }

$L \rightarrow S$ { $L.\text{nextlist} := S.\text{nextlist}$ }